

Reproducibility Guide: 5G Network Benchmarking Study

This document provides step-by-step instructions to reproduce the main results reported in Table 4 of the paper "msData: A Millisecond-Resolution Network Dataset for Advancing Time Series Foundation Models".

Quick Start

Follow the steps below to reproduce the main experiments (Table 4) and collect the final results.

Important: All main experiments in this repository use the filtered dataset `youtube_static.csv`.

1. Create the required conda environments

```
conda env create -f environment.yml
conda activate tf-env

conda env create -f arf_environment.yml
conda activate river-env
```

For the main experiments in this repository, the dataset used is: `data/filtered/youtube_static.csv`

2. Run the notebook-based experiments

For univariate experiments:

```
conda activate tf-env
scripts\reproduce_univariate.bat
```

For multivariate experiments:

```
conda activate tf-env
scripts\reproduce_multivariate.bat
```

3. Run ARF and OLR separately

ARF:

```
conda activate river-env
python scripts\ARF_Univariate.py --seeds 42
python scripts\ARF_Multivariate.py --seeds 42
```

OLR:

```
conda activate river-env
python scripts\OLR_Multivariate.py
python scripts\OLR_Univariate.py
```

4. Output Location

The saved metrics are written to: results/metrics/

Repository Structure

The repository is organized as follows:

- **README.md**: short project overview and quick entry point
- **REPRODUCIBILITY.md**: detailed step-by-step reproduction guide
- **environment.yml**: conda environment for TensorFlow/foundation-model experiments
- **arf_environment.yml**: conda environment for River-based streaming models
- **data/raw/**: original raw dataset (empty for now)
- **data/filtered/**: filtered datasets generated before model execution
- **notebooks/**: experiment notebooks for all models
- **scripts/**: helper scripts, Python runners, and Windows batch files
- **results/metrics/**: saved model metrics

```
tmlr-reproducibility/
├── README.md
├── REPRODUCIBILITY.md
├── environment.yml
├── arf_environment.yml
├── data/
│   ├── raw/
│   └── filtered/
├── notebooks/
├── scripts/
└── results/
    ├── metrics/
    ├── forecasts/
    └── figures/
```

Environment Map

Different models require different software environments.

tf-env

Use this environment for:

- Random Forest
- XGBoost
- Naive
- TTM
- Chronos
- Lag-Llama
- iTransformer
- PatchTST

river-env

Use this environment for:

- Adaptive Random Forest (ARF)
- Online Linear Regression (OLR)

Complete Environment Setup

TensorFlow environment (most models)

```
conda env create -f environment.yml
conda activate tf-env
```

ARF/OLR (streaming/online learning)

For Adaptive Random Forest, first ensure Python 3 is installed. Then, install the River library and create the conda environment using `arf_environment.yml`

```
conda env create -f arf_environment.yml
conda activate river-env
```

Model-Specific One-Time Setup:

The following TSFMs require additional one-time setup before the experiment notebooks can be executed:

- Chronos

- TTM
- Lag-Llama

Chronos

To set up Chronos, follow these instructions from the official GitHub repository:

```
conda activate tf-env
!pip install autogluon
If keras3 is installed, then install the backwards-compatible tf-keras package
with `!pip install tf-keras`.
```

TTM

To set up TTM, follow these instructions from the official GitHub repository:

```
conda activate tf-env
# Clone the ibm/tsfm
! git clone https://github.com/ibm-granite/granite-tsfm.git
! ls

# Change directory. Move inside the tsfm repo.
%cd granite-tsfm
! ls

# Relax requirement for python version < 3.12
! sed -i.orig 's/3\..12/3.13/g' pyproject.toml

# Install the tsfm library
#! pip install ".[notebooks]"
#! python3 -m pip install ".[notebooks]"
! pip3 install ".[notebooks]"
```

Lag-Llama

To set up Lag-Llama, follow these instructions from the official GitHub repository:

```
!git clone -b update-gluonts https://github.com/time-series-foundation-models/lag-llama/
cd lag-llama
!pip install -r requirements.txt
!pip install -U torch torchvision
!huggingface-cli download time-series-foundation-models/Lag-Llama lag-llama.ckpt -
```

```
-local-dir lag-llama  
cd lag-llama
```

Expected versions (minimum):

- Python: 3.9+
- pandas: 2.1+
- scikit-learn: 1.5+
- tensorflow: 2.17+
- numpy: 1.24+
- torch: 2.10.0+
- scipy: 1.10+
- xgboost: 3.0 +
- transformers: 4.50+
- autogluon: 1.1+
- tf-keras:2.15+
- granite-tsfm:0.1+
- matplotlib:3.8+
- river:0.20+

Verify Installation

Run:

```
python test_imports.py
```

Dataset Preparation

1. Filter Dataset

Before running any model, generate the filtered datasets. This creates two CSV files:

- `youtube_static.csv`
- `Web_Browsing_train.csv`

```
python scripts/filter_data.py
```

Note: The main experiments reported in this reproducibility guide use `youtube_static.csv`.

2. Verify Dataset

Run:

```
python scripts/verify_dataset.py
```

Reproducing Experiments

Main Results (Table 4: Performance Metrics)

To reproduce the main experiments, use the commands below.

Windows Note

This repository provides Windows batch files (`.bat`) for notebook execution.

Run them from Anaconda Prompt or Command Prompt after activating the correct conda environment.

Notebook-based experiments (tf-env)

Run the multivariate notebook group using the appropriate batch file:

```
conda activate tf-env
scripts\reproduce_multivariate.bat
```

Run the univariate notebook group using the appropriate batch file:

```
conda activate tf-env
scripts\reproduce_univariate.bat
```

The ARF and OLR experiments are run separately in the River/streaming-model environment. Run ARF:

```
conda activate river-env
python scripts\ARF_Univariate.py --seeds 42
python scripts\ARF_Multivariate.py --seeds 42
```

Run OLR:

```
conda activate river-env
python scripts\OLR_Univariate.py
python scripts\OLR_Multivariate.py
```

Lag-Llama

Lag-Llama requires a one-time external setup before the zero-shot or fine-tuning notebook can be executed, as mentioned above (Installation). After completing the one-time setup, run the Lag-Llama notebook.

Outputs

All Model Outputs are saved in: results/metrics/

Collecting Results

After all models have finished running, combine the saved metrics into a single summary table:

```
python scripts\collect_results.py
```